

# Safe NixOS Fresh-Install Password Provisioning

## ULW-Research Synthesis: Safe Fresh-Install Password Provisioning for `nix-config`

Workers: 21 agents + 1 orchestrator lane · Research waves: 3 · Primary  
source families: 9 · Executed verification suites: 6

### Executive summary

Do **not** add a literal “default password” with `initialPassword`, and do **not** commit a reusable `initialHashedPassword`/`hashedPassword` to this public repository. NixOS warns that plaintext password options enter the world-readable Nix store; a published hash is still a reusable verifier that supports offline guessing. [Source 1] [Source 5] [Source 6]

For this repository, the safest practical design is a **unique password per installation**, represented as an interactively generated `yescrypt` hash, delivered by `nixos-anywhere --extra-files` to a root-only persistent file:

```
/var/lib/nixos-bootstrap/mei-password.hash
```

Configure `users.users.mei.hashedPasswordFile` to that path while keeping the current classic backend and `users.mutableUsers = true`. Validate the file before the `users` activation fragment, consume it for the newly created account, then replace the extra bootstrap copy with the locked sentinel `!` after `users` activation. The validator accepts that sentinel only when `/etc/shadow` already contains an unlocked password for `mei`; it therefore fails closed if a sentinel is accidentally staged on a fresh or locked machine. On later rebuilds, `mutable-user`

semantics preserve the actual password in `/etc/shadow`, while the sentinel prevents stale verifier reuse and avoids missing-file warnings. [Source 1] [Source 2] [Source 9]

This recommendation is repo-specific: LightDM requires a local login credential, normal wheel sudo requires a password, the configured SSH key does not match this workstation, and the Disko layout has no FDE. A local password improves login/sudo usability but does **not** protect unencrypted data at rest. See `hosts/nixos/default.nix:272-302` and `modules/nixos/disk-config.nix:4-33`. [Source 8]

## Recommended configuration shape

Add the following concepts to `hosts/nixos/default.nix`; this is a researched candidate, not an applied change:

```
let
  user = "mei";
  bootstrapHashFile = "/var/lib/nixos-bootstrap/mei-
    password.hash";
  # existing keys declaration remains
in {
  users.mutableUsers = true;
  users.users.${user}.hashedPasswordFile = bootstrapHashFile;

  system.activationScripts.bootstrapPasswordHash = {
    text = ''
      hash_file=${lib.escapeShellArg bootstrapHashFile}
      fail() { echo "bootstrap password hash validation failed:
        $1" >&2; exit 1; }

      test -f "$hash_file" || fail "missing $hash_file"
      test "${pkgs.coreutils}/bin/stat -c '%U:%G:%a'
        "$hash_file")" = \
        "root:root:600" || fail "expected root:root mode 0600"
      test -s "$hash_file" || fail "expected non-empty file"
      last_byte="${pkgs.coreutils}/bin/tail -c 1 "$hash_file" \
        | ${pkgs.coreutils}/bin/od -An -tuC \
        | ${pkgs.coreutils}/bin/tr -d '[:space:]'"
      test "$last_byte" = 10 || fail "expected one newline-
        terminated line"
      test "${pkgs.coreutils}/bin/wc -l < "$hash_file")" -eq 1
        || \
        fail "expected exactly one newline-terminated line"
      if ${pkgs.gnugrep}/bin/grep -qx '!' "$hash_file"; then
```

```

# Reject a sentinel on a fresh machine or for a locked
account.
test -r /etc/shadow || fail "cannot verify consumed
sentinel"
${pkgs.gawk}/bin/awk -F: -v target_user=$
{lib.escapeShellArg user} '
    $1 == target_user {
        found = 1
        if ($2 != "" && $2 !~ /^[!*/]) unlocked = 1
    }
    END { exit !(found && unlocked) }
' /etc/shadow || fail \
    "consumed sentinel requires an existing unlocked
password"
else
    ${pkgs.gnugrep}/bin/grep -Eqx \
        '^$y\[./A-Za-z0-9]+\\[./A-Za-z0-9]{0,86}\\[./A-Za-
z0-9]{43}$' \
        "$hash_file" || fail "expected yescrypt hash"
fi
'';
};

system.activationScripts.users.deps = [
    "bootstrapPasswordHash" ];

system.activationScripts.consumeBootstrapPassword = {
    deps = [ "users" ];
    text = ''
        hash_file=${lib.escapeShellArg bootstrapHashFile}
        if ! ${pkgs.gnugrep}/bin/grep -qx '!' "$hash_file"; then
            tmp="$hash_file.tmp.$$"
            trap 'rm -f "$tmp"' EXIT
            ${pkgs.coreutils}/bin/printf '!\\n' > "$tmp"
            ${pkgs.coreutils}/bin/chown root:root "$tmp"
            ${pkgs.coreutils}/bin/chmod 0600 "$tmp"
            ${pkgs.coreutils}/bin/mv -f "$tmp" "$hash_file"
            trap - EXIT
        fi
    '';
};

assertions = [
    {
        assertion = config.users.mutableUsers;
        message = "bootstrap password sentinel requires mutable
users";
    }
];

```

```

    }
    {
      assertion = !config.systemd.sysusers.enable && !
        config.services.userborn.enable;
      message = "bootstrap password ordering requires classic
        users activation";
    }
  ];
}

```

The isolated overlay evaluated successfully, preserved mei's groups and SSH key, produced a valid system derivation, and emitted activation order `bootstrapPasswordHash` → `users` → `consumeBootstrapPassword`.  
[Verification V1]

## Safe per-install workflow in Fish

Run this on the working computer from the patched temporary clone. It never puts the plaintext password in `argv`, an environment variable, `Git`, or the Nix store:

```

function install_with_bootstrap_password
  set runtime "$XDG_RUNTIME_DIR"
  test -n "$runtime"; or return 1
  findmnt -no FSTYPE --target "$runtime" | string match -qr
    '^tmpfs$'; or return 1

  set stage (mktemp -d "$runtime/nixos-extra.XXXXXXXXXX"); or
return 1
  function __cleanup_nixos_password_stage --inherit-variable
stage
    command rm -rf -- "$stage"
    functions --erase __cleanup_nixos_password_exit \
      __cleanup_nixos_password_hup
__cleanup_nixos_password_int \
      __cleanup_nixos_password_term
__cleanup_nixos_password_stage \
      install_with_bootstrap_password
    end
    function __cleanup_nixos_password_exit --on-event fish_exit --
inherit-variable stage
      command rm -rf -- "$stage"
    end
    function __cleanup_nixos_password_hup --on-signal HUP

```

```

        __cleanup_nixos_password_stage
        exit 129
    end
    function __cleanup_nixos_password_int --on-signal INT
        __cleanup_nixos_password_stage
        exit 130
    end
    function __cleanup_nixos_password_term --on-signal TERM
        __cleanup_nixos_password_stage
        exit 143
    end

    command install -d -m 700 "$stage/var/lib/nixos-bootstrap"; or
begin
    __cleanup_nixos_password_stage
    return 1
end

    # Prompts privately on the controlling terminal; stdout goes
    directly to the file.
    nix shell nixpkgs#mkpasswd --command mkpasswd --
method=yescrypt \
    > "$stage/var/lib/nixos-bootstrap/mei-password.hash"; or
begin
    __cleanup_nixos_password_stage
    return 1
end
    command chmod 600 "$stage/var/lib/nixos-bootstrap/mei-
password.hash"; or begin
    __cleanup_nixos_password_stage
    return 1
end

    test (command tail -c 1 "$stage/var/lib/nixos-bootstrap/mei-
password.hash" \
        | command od -An -tuC | string trim) = 10; and \
    test (command wc -l < "$stage/var/lib/nixos-bootstrap/mei-
password.hash" \
        | string trim) -eq 1; and \
    command grep -Eqx '^\\$y\\$[./A-Za-z0-9]+\\[./A-Za-z0-9]{0,86}\\$
[./A-Za-z0-9]{43}\\$' \
        "$stage/var/lib/nixos-bootstrap/mei-password.hash"; or begin
    __cleanup_nixos_password_stage
    return 1

```

```

end

nix run github:nix-community/nixos-anywhere -- \
  --flake ".#x86_64-linux" \
  --target-host "root@$TARGET" \
  --build-on local \
  --extra-files "$stage" \
  --option max-jobs 1 \
  --option cores 1

set rc $status
__cleanup_nixos_password_stage
return $rc
end

install_with_bootstrap_password

```

Use a unique strong password for each installation. Do not enable `nixos-anywhere --debug`, shell tracing, clipboard capture, or terminal recording while generating/staging it. `--extra-files` preserves modes and extracts as root. [Source 3] [Source 7]

## Why `/var/lib`, not `/run`

A wave-1 claim proposed `/run`, but two later source chains refuted it. `nixos-install` invokes the boot action without normal activation; on first boot NixOS mounts runtime tmpfs at `/run` before the users activation reads `hashedPasswordFile`. Therefore an on-disk file staged at `/mnt/run/...` is unavailable at the relevant read. The `nixos-anywhere` integration suite demonstrates that `/var/lib` content persists through install and reboot. [Source 3] [Source 4]

## Security and lifecycle findings

| Candidate                                    | Result                      | Reason                                                                   |
|----------------------------------------------|-----------------------------|--------------------------------------------------------------------------|
| <code>initialPassword = "..."</code>         | Reject                      | Plaintext enters Git/store; NixOS explicitly warns it is world-readable. |
| <code>Committed initialHashedPassword</code> | Reject for this public repo | Functional, but publishes a reusable offline-verifiable                  |

| Candidate                              | Result                     | Reason                                                                                                                                            |
|----------------------------------------|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Persistent external hashedPasswordFile | Acceptable fallback        | hash and retains it in history.<br>Keeps hash outside Git/store but duplicates a stale verifier indefinitely.                                     |
| External hash replaced by !            | Recommended now            | Per-install design evaluates correctly; duplicate verifier is removed and later mutable password is preserved. End-to-end login remains untested. |
| Agenix encrypted hash                  | Recommended later, not now | Strong declarative model, but current repo has no first-boot target identity provisioning and would create a bootstrap loop.                      |
| SSH-only                               | Conditional                | No password exposure, but current wrong-key incident, LightDM, and passworded sudo make it unsuitable as the sole laptop access path.             |

users.mutableUsers must remain true for the sentinel design. If changed to false, later activation would declaratively apply ! and lock the account. Password rotation on an installed machine remains passwd mei; changing the installer hash does not rotate an existing mutable user. [Source 1] [Source 2]

# SSH and disk caveats

Do not harden SSH to key-only until the repository contains and tests the correct public key. The configured fingerprint is SHA256:QMzHpX...; this workstation's id\_ed25519.pub is SHA256:Nn1Zq... After correcting and testing the key, a separate hardening change should consider:

```
services.openssh.settings = {  
  PasswordAuthentication = false;  
  KbdInteractiveAuthentication = false;  
  PermitRootLogin = "no";  
};
```

If inbound SSH is unnecessary, disabling OpenSSH is simpler. Independently, the current plaintext ext4 Disko layout means neither the local password nor encrypted secret delivery protects data from offline disk access; FDE is separate future work. [Source 8]

## Verified claims

| Claim                                                                                                     | Verdict   | Artifact                        |
|-----------------------------------------------------------------------------------------------------------|-----------|---------------------------------|
| Candidate option and activation ordering evaluate                                                         | Confirmed | verify-repo-overlay-gate-fix.md |
| Groups/authorized keys preserved in overlay                                                               | Confirmed | verify-repo-overlay-gate-fix.md |
| Exact emitted validator accepts yescrypt and accepts a sentinel only for an existing unlocked account     | Confirmed | verify-emitted-candidate.md     |
| Exact emitted validator rejects missing, empty, multiline, unterminated, malformed, and wrong-mode inputs | Confirmed | verify-emitted-candidate.md     |
|                                                                                                           | Confirmed |                                 |



| Claim                                                                                                                                      | Verdict   | Artifact                        |
|--------------------------------------------------------------------------------------------------------------------------------------------|-----------|---------------------------------|
| Exact emitted consumer replaces the verifier with !                                                                                        |           | verify-emitted-candidate.md     |
| Candidate build dry-run succeeds                                                                                                           | Confirmed | verify-repo-overlay-gate-fix.md |
| Exact documented Fish workflow cleans up on success, generator/installer failure, and HUP/INT/TERM while preserving status                 | Confirmed | verify-exact-fish-workflow.md   |
| Tracked repository contains no quoted/unquoted Nix password-option assignment, enumerated modular/PHC password hash, or private-key marker | Confirmed | verify-tracked-secret-scan.md   |
| Exact mkpasswd package exposes yescrypt                                                                                                    | Confirmed | verify-mkpasswd.md              |
| /run staging works                                                                                                                         | Refuted   | verify-run-path.md              |

## Epistemic instrumentation

- Intent/reality closure: intent-diff.md
- Verified claim allowlist and counterevidence: claim-graph.md
- Independent observations and temporal validity: observation-manifest.md
- Proof-cost decisions: verification-economics.md
- Replaced causal theory (/run  $\rightarrow$  /var/lib): cause-disappearance.md
- Independent anti-slop/programming rejection, fixes, and approval: anti-slop-programming-review.md

Production remains unchanged because the user requested research, not implementation. The proposed design resolves the lockout in isolated evaluation only.

# Contradictions

1. **/run viability:** refuted after install/boot/initrd tracing and an upstream `/var/lib` integration test.
2. **SSH-only vs local password:** resolved conditionally. SSH-only is strongest against password exposure, but this laptop's local greeter/sudo needs and wrong-key incident favor a unique injected local password plus independently verified SSH keys.
3. **Persistent hash vs encrypted agenix:** agenix is stronger once per-host identity provisioning exists; today it is more complex and can fail before user creation.

# Gaps

- No destructive reinstall or real PAM/LightDM login was performed during research.
- No full NixOS VM boot was run; the candidate was evaluated/built and validator logic executed independently.
- Actual laptop `/etc/shadow`, SSH key state, and block-device encryption were not remotely inspected after recovery.
- FDE, Secure Boot, TPM, and root-SSH policy remain separate decisions.

# Expansion trace

- Wave 1: seven axes established repo state, option semantics, secret threats, installer behavior, and candidate designs.
- Wave 2: six axes closed backend/FDE/key state, secret exposure, agenix identity, one-shot behavior, and laptop policy; discovered the `/run` contradiction.
- Wave 3: six counter-search/design lanes resolved `/run`, ranked lifecycle designs, proved activation feasibility, and adversarially reviewed the recommendation.
- Convergence: all password-mechanism leads are closed or reduced to explicit implementation/testing work; no unchecked research lead remains.

# Sources

1. [Nixpkgs user/password option implementation](#)
2. [Nixpkgs mutable user activation implementation](#)

3. nixos-anywhere transfer/install implementation
4. nixos-anywhere persistent extra-files integration test
5. Nix manual: secrets and the store
6. NIST password verifier guidance
7. libxcrypt yescrypt format and recommendation
8. Local repository evidence: `hosts/nixos/default.nix`, `modules/nixos/secrets.nix`, `modules/nixos/disk-config.nix`, `docs/service-notes/nixos-anywhere-disko-install.md`
9. shadow(5) password-lock sentinel semantics